

# **MECHATRONICS AND IOT**

## **ASSIGNMENT 6**

**Design and develop a Wireless  
Environmental Parameter Monitoring  
System for Smart  
Industries.**

### **TEAM MEMBERS:**

**KOWSHIK K 24MER025**

**NITHISH J 24MER037**

**KAVIN KRISHNA D S - 24MER024**

**LOGESH S 24MER027**

# 1. Project Overview

The **Wireless Environmental Parameter Monitoring System** is an IoT-based system designed to continuously monitor and record important environmental conditions in industrial environments. Industries such as manufacturing plants, workshops, chemical industries, and warehouses may experience **high temperature, excessive humidity, dust, smoke, and poor air quality**, which can affect workers' health, machine performance, and overall production efficiency.

The proposed system uses an **ESP32/ESP8266 microcontroller** as the main control unit. Sensors are connected to the controller to measure parameters such as **temperature, humidity, and air quality** at regular intervals. The collected sensor data is processed by the microcontroller and transmitted wirelessly through **Wi-Fi** to an IoT platform or online dashboard.

The dashboard allows the user to **monitor environmental conditions remotely in real time** without being physically present at the industrial location. If any parameter exceeds its predefined safe limit, the system can generate an **alert or warning** to notify the user. A local display can also be used to show the current sensor readings and system status.

This system helps industries in **early detection of unsafe environmental conditions**, reduces the need for continuous manual monitoring, improves workplace safety, and supports better maintenance of industrial equipment. The system can also be expanded in the future by adding **gas sensors, dust sensors, data logging, automatic ventilation control, and mobile notifications**.

## 2. Problem Statement

In many industries, environmental parameters are checked manually or at fixed time intervals. This method may not detect sudden changes in temperature, humidity, or air quality.

Poor environmental conditions can:

- ✓ Affect worker safety and comfort.
- ✓ Cause overheating of machines.
- ✓ Reduce equipment life.
- ✓ Affect product quality.

- ✓ Increase the risk of harmful gas exposure.
- ✓ Cause delays because abnormal conditions are detected late.

Therefore, a **continuous wireless monitoring system** is required to provide real-time information and alerts.

### 3. Proposed Solution

The proposed **Wireless Environmental Parameter Monitoring System** uses an **ESP32 NodeMCU** as the main controller because it has built-in Wi-Fi connectivity and sufficient processing capability for IoT applications. A **DHT11/DHT22 sensor** is used to measure the surrounding **temperature and humidity**, while an **MQ-135 air-quality sensor** detects changes in air quality caused by gases, smoke, and other pollutants.

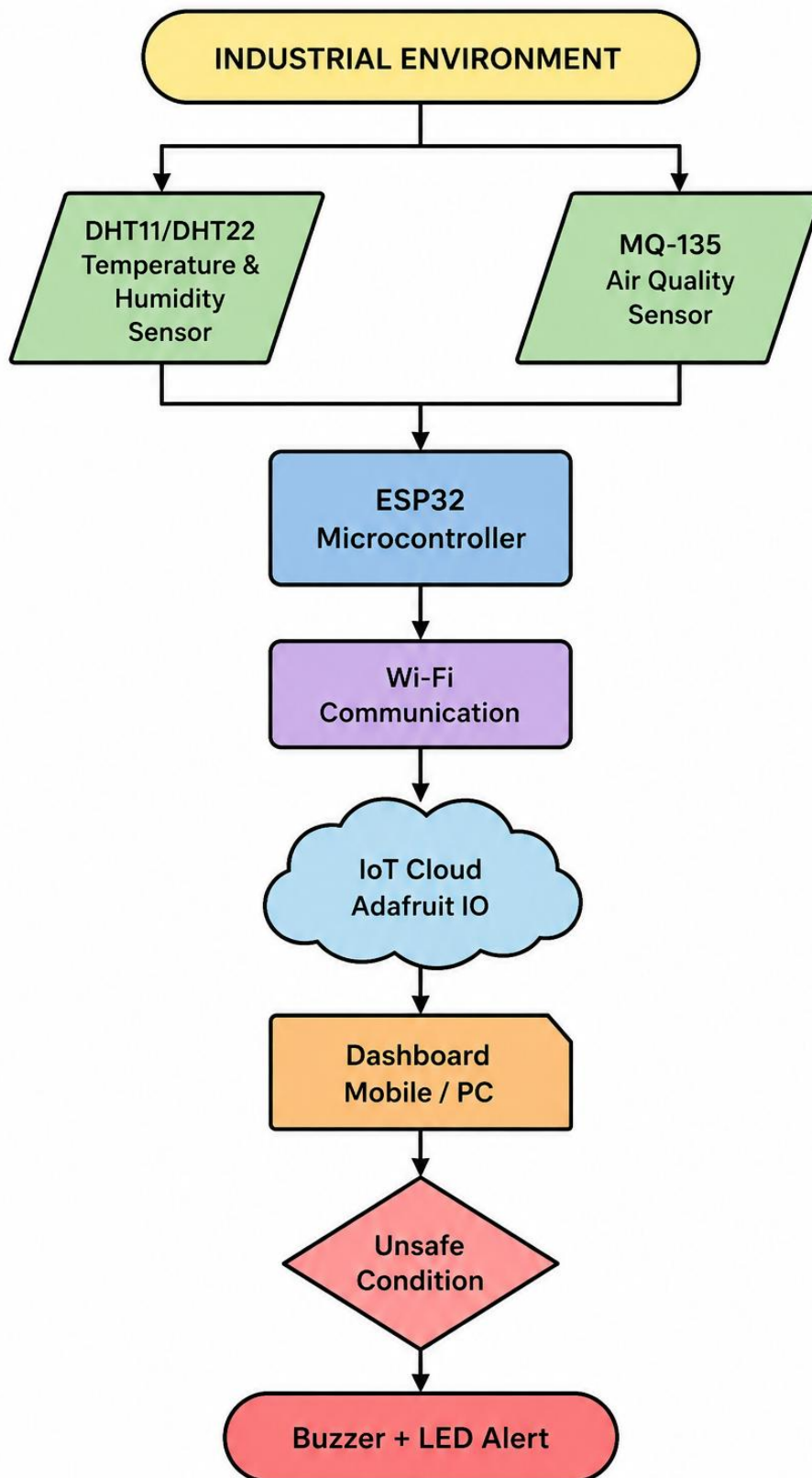
The ESP32 continuously collects data from the sensors and processes the readings according to predefined safety limits. The measured values are transmitted wirelessly through Wi-Fi to an IoT platform such as **Adafruit IO**, where the data can be displayed using gauges, charts, and indicators. This allows the user to monitor the industrial environment remotely using a **mobile phone or computer**.

The proposed system provides a **low-cost, compact, and easily expandable solution** for industrial environmental monitoring. Additional sensors and control devices can be integrated in the future according to the requirements of the industry.

### Main Features

- ✓ **Real-time temperature monitoring** using DHT11/DHT22.
- ✓ **Real-time humidity monitoring** for identifying excessive moisture conditions.
- ✓ **Air-quality monitoring** using the MQ-135 sensor.
- ✓ **Wireless data transmission** through the ESP32's built-in Wi-Fi.
- ✓ **Remote IoT dashboard** using Adafruit IO.
- ✓ **Local warning system** using buzzer and LED.
- ✓ **Continuous monitoring** at regular time intervals.
- ✓ **Predefined safety limits** for identifying abnormal conditions.
- ✓ **Historical data monitoring** through IoT graphs and charts.

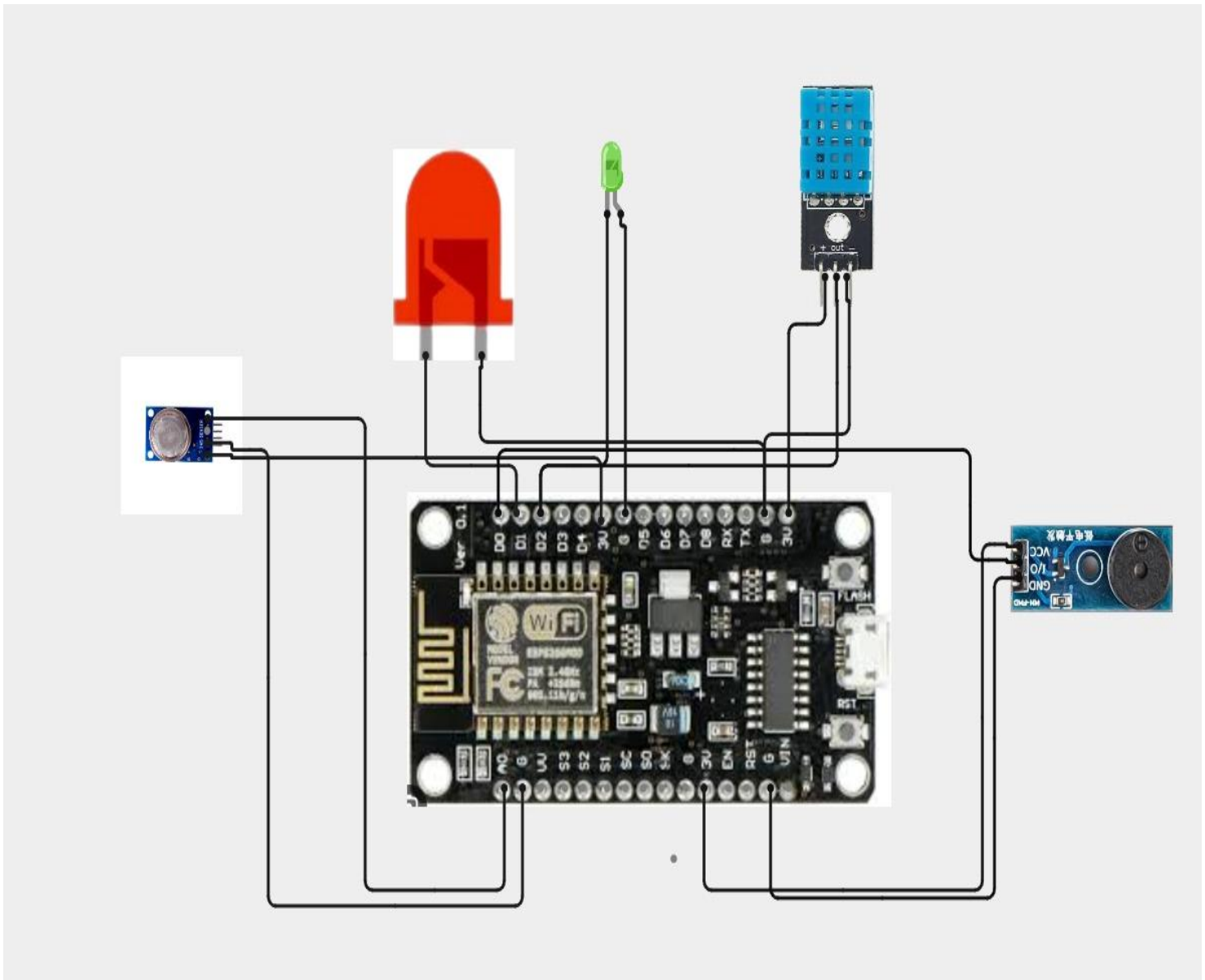
## 4. System Architecture



## 5. Circuit Table

| S.No | Component   | Pin     | ESP32 Pin | Function                    |
|------|-------------|---------|-----------|-----------------------------|
| 1    | DHT11/DHT22 | VCC     | 3.3V      | Sensor power                |
| 2    | DHT11/DHT22 | GND     | GND       | Ground                      |
| 3    | DHT11/DHT22 | DATA    | GPIO 4    | Temperature & humidity data |
| 4    | MQ-135      | VCC     | 5V/VIN    | Sensor power                |
| 5    | MQ-135      | GND     | GND       | Ground                      |
| 6    | MQ-135      | AO      | GPIO 34   | Air-quality reading         |
| 7    | Buzzer      | +       | GPIO 25   | Warning sound               |
| 8    | Buzzer      | –       | GND       | Ground                      |
| 9    | LED         | Anode   | GPIO 26   | Warning indication          |
| 10   | LED         | Cathode | GND       | Ground                      |

## 6. Circuit Design



## 7. Algorithm

### Step-by-Step Algorithm

- ✓ **Start the system**  
Power ON the ESP32 and all connected sensors.
- ✓ **Initialize the components**  
Initialize the DHT11/DHT22, MQ-135, buzzer, LED, and required ESP32 GPIO pins.

✓ **Establish Wi-Fi connection**

The ESP32 connects to the available Wi-Fi network. Once connected, it establishes communication with the IoT platform such as Adafruit IO.

✓ **Read temperature and humidity**

The DHT11/DHT22 measures the surrounding temperature and humidity and sends the readings to the ESP32.

✓ **Read air-quality value**

The MQ-135 provides an analog output corresponding to changes in air quality. The ESP32 reads this value through its ADC pin.

✓ **Process the sensor data**

The ESP32 processes the received values and prepares them for comparison and display.

✓ **Compare with threshold values**

The measured temperature, humidity, and air-quality values are compared with predefined threshold limits.

✓ **Check safety condition**

- If all values are within the selected safe limits, the system displays **SAFE** and keeps the buzzer and LED OFF.
- If any parameter exceeds its threshold, the system identifies the condition as **UNSAFE**.

✓ **Activate warning system**

During an unsafe condition, the ESP32 turns **ON the buzzer and LED** to provide a local warning.

✓ **Upload data to IoT dashboard**

Temperature, humidity, air-quality readings, and safety status are transmitted through Wi-Fi to the IoT dashboard.

✓ **Display and monitor data**

The user can view the current environmental values and safety status remotely using a mobile phone or computer.

✓ **Repeat continuously**

After a short delay, the ESP32 repeats the sensing, comparison, alert, and data-upload process continuously.

## 8. Coding

```
// =====
```

```
// WIRELESS ENVIRONMENTAL PARAMETER MONITORING SYSTEM
```

**// ESP8266 + DHT11 + MQ135 + Buzzer + ThingSpeak**

**// =====**

**#include <ESP8266WiFi.h>**

**#include <ThingSpeak.h>**

**#include <DHT.h>**

**// =====**

**// WIFI DETAILS**

**// =====**

**#define WIFI\_SSID "Luky003"**

**#define WIFI\_PASSWORD "kowshik03"**

**// =====**

**// THINGSPEAK DETAILS**

**// =====**

**unsigned long channelID = 3459014;**

**const char\* writeAPIKey = "HPFMS1KQ7C86UZFC";**

**WiFiClient client;**

**// =====**

**// PIN DEFINITIONS**



```
// =====

#define DHT_PIN 4 // D2 = GPIO4

#define MQ135_PIN A0

#define BUZZER_PIN 14 // D5 = GPIO14

// =====

// DHT SENSOR TYPE

// =====

#define DHTTYPE DHT11

// For DHT22 use:

// #define DHTTYPE DHT22

DHT dht(DHT_PIN, DHTTYPE);

// =====

// AIR QUALITY THRESHOLDS

// =====

#define GOOD_LIMIT 399

#define MODERATE_LIMIT 599

// =====
```

```
// TIMER
```

```
// =====
```

```
unsigned long previousMillis = 0;
```

```
const unsigned long interval = 60000; // 60 seconds
```

```
// =====
```

```
// SETUP
```

```
// =====
```

```
void setup()
```

```
{
```

```
  Serial.begin(115200);
```

```
  delay(1000);
```

```
  Serial.println();
```

```
  Serial.println("=====");
```

```
  Serial.println(" WIRELESS ENVIRONMENT MONITORING SYSTEM");
```

```
  Serial.println(" ESP8266 + DHT11 + MQ135 + BUZZER");
```

```
  Serial.println(" THINGSPEAK CLOUD");
```

```
  Serial.println("=====");
```

```
// Start DHT
```

```
dht.begin();
```

```
// Buzzer setup
```

```
pinMode(BUZZER_PIN, OUTPUT);
```

```
digitalWrite(BUZZER_PIN, LOW);
```

```
// =====
```

```
// CONNECT TO WIFI
```

```
// =====
```

```
Serial.println("Connecting to WiFi...");
```

```
WiFi.mode(WIFI_STA);
```

```
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
```

```
int attempts = 0;
```

```
while (WiFi.status() != WL_CONNECTED && attempts < 40)
```

```
{
```

```
    delay(500);
```

```
    Serial.print(".");
```

```
    attempts++;
```

```
}
```

```
Serial.println();
```

```
if (WiFi.status() == WL_CONNECTED)
```

```

{
    Serial.println("WiFi Connected!");

    Serial.print("IP Address: ");
    Serial.println(WiFi.localIP());

    Serial.print("Signal Strength: ");
    Serial.print(WiFi.RSSI());
    Serial.println(" dBm");

    // Start ThingSpeak
    ThingSpeak.begin(client);
}

else
{
    Serial.println("WiFi Connection Failed!");
}

Serial.println("-----");
}

// =====

// LOOP

// =====

```

```
void loop()

{

    // =====

    // READ DHT11

    // =====

    float temperature = dht.readTemperature();

    float humidity = dht.readHumidity();

    if (isnan(temperature) || isnan(humidity))

    {

        Serial.println("ERROR: DHT sensor reading failed!");

        delay(2000);

        return;

    }

    // =====

    // READ MQ135

    // =====

    int airQualityValue = analogRead(MQ135_PIN);

    // =====

    // AIR QUALITY STATUS

    // =====
```

```
String airStatus;
```

```
if (airQualityValue <= GOOD_LIMIT)
```

```
{
```

```
    airStatus = "GOOD";
```

```
    digitalWrite(BUZZER_PIN, LOW);
```

```
}
```

```
else if (airQualityValue <= MODERATE_LIMIT)
```

```
{
```

```
    airStatus = "MODERATE";
```

```
    digitalWrite(BUZZER_PIN, LOW);
```

```
}
```

```
else
```

```
{
```

```
    airStatus = "POOR";
```

```
    digitalWrite(BUZZER_PIN, HIGH);
```

```
}
```

```
// =====
```

```
// SERIAL MONITOR
```

```
// =====
```

```
Serial.println();
```

```
Serial.println("----- SENSOR DATA -----");
```

```
Serial.print("Temperature    : ");
```

```
Serial.print(temperature);
```

```
Serial.println(" °C");
```

```
Serial.print("Humidity      : ");
```

```
Serial.print(humidity);
```

```
Serial.println(" %");
```

```
Serial.print(" Air Quality Value : ");
```

```
Serial.println(airQualityValue);
```

```
Serial.print(" Air Quality    : ");
```

```
Serial.println(airStatus);
```

```
Serial.print("Buzzer        : ");
```

```
if (airQualityValue > MODERATE_LIMIT)
```

```
{
```

```
    Serial.println("ON");
```

```
}
```

```
else
```

```
{  
  
    Serial.println("OFF");  
  
}  
  
Serial.println("-----");  
  
// =====  
  
// SEND DATA TO THINGSPEAK EVERY 60 SECONDS  
  
// =====  
  
unsigned long currentMillis = millis();  
  
if (currentMillis - previousMillis >= interval)  
{  
    previousMillis = currentMillis;  
  
    if (WiFi.status() == WL_CONNECTED)  
    {  
        Serial.println("Sending data to ThingSpeak...");  
  
        // Field 1 = Temperature  
        ThingSpeak.setField(1, temperature);  
  
        // Field 2 = Humidity  
        ThingSpeak.setField(2, humidity);  
    }  
}
```



```
// Field 3 = Air Quality Value  
ThingSpeak.setField(3, airQualityValue);
```

```
// Field 4 = Air Status
```

```
// GOOD = 0
```

```
// MODERATE = 1
```

```
// POOR = 2
```

```
int statusValue;
```

```
if (airStatus == "GOOD")
```

```
{
```

```
    statusValue = 0;
```

```
}
```

```
else if (airStatus == "MODERATE")
```

```
{
```

```
    statusValue = 1;
```

```
}
```

```
else
```

```
{
```

```
    statusValue = 2;
```

```
}
```

```
ThingSpeak.setField(4, statusValue);
```

```
// Send all fields

int response = ThingSpeak.writeFields(channelID, writeAPIKey);

if (response == 200)
{
    Serial.println("ThingSpeak: Data sent successfully!");
}
else
{
    Serial.print("ThingSpeak Error: ");
    Serial.println(response);
}

Serial.println("-----");
}

else
{
    Serial.println("WiFi disconnected!");
    Serial.println("Reconnecting...");
    WiFi.disconnect();
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
}
}

delay(2000);
}
```

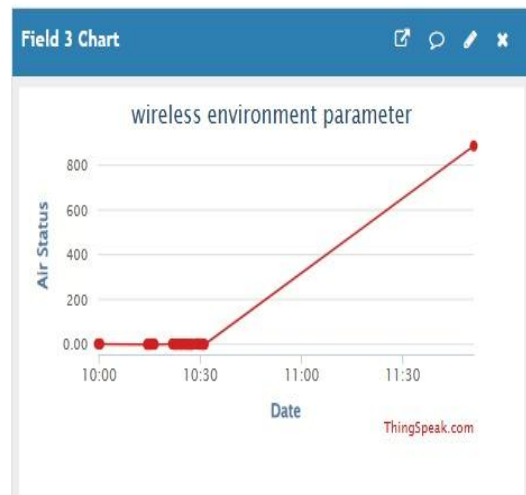
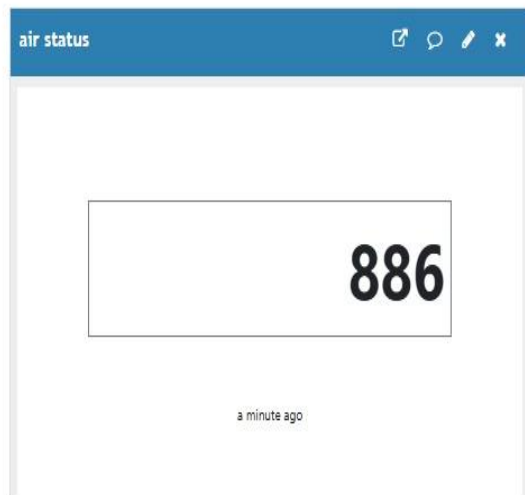
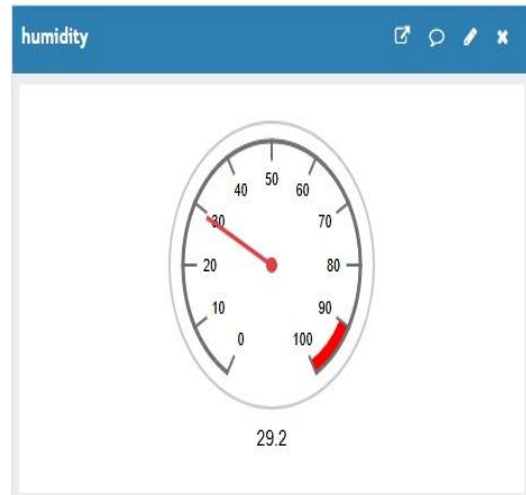
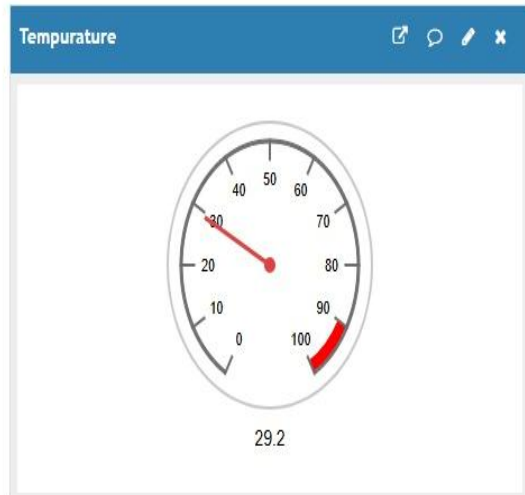
## 9. System Working

### Channel Stats

Created: [a day ago](#)

Last entry: [about a minute ago](#)

Entries: 80



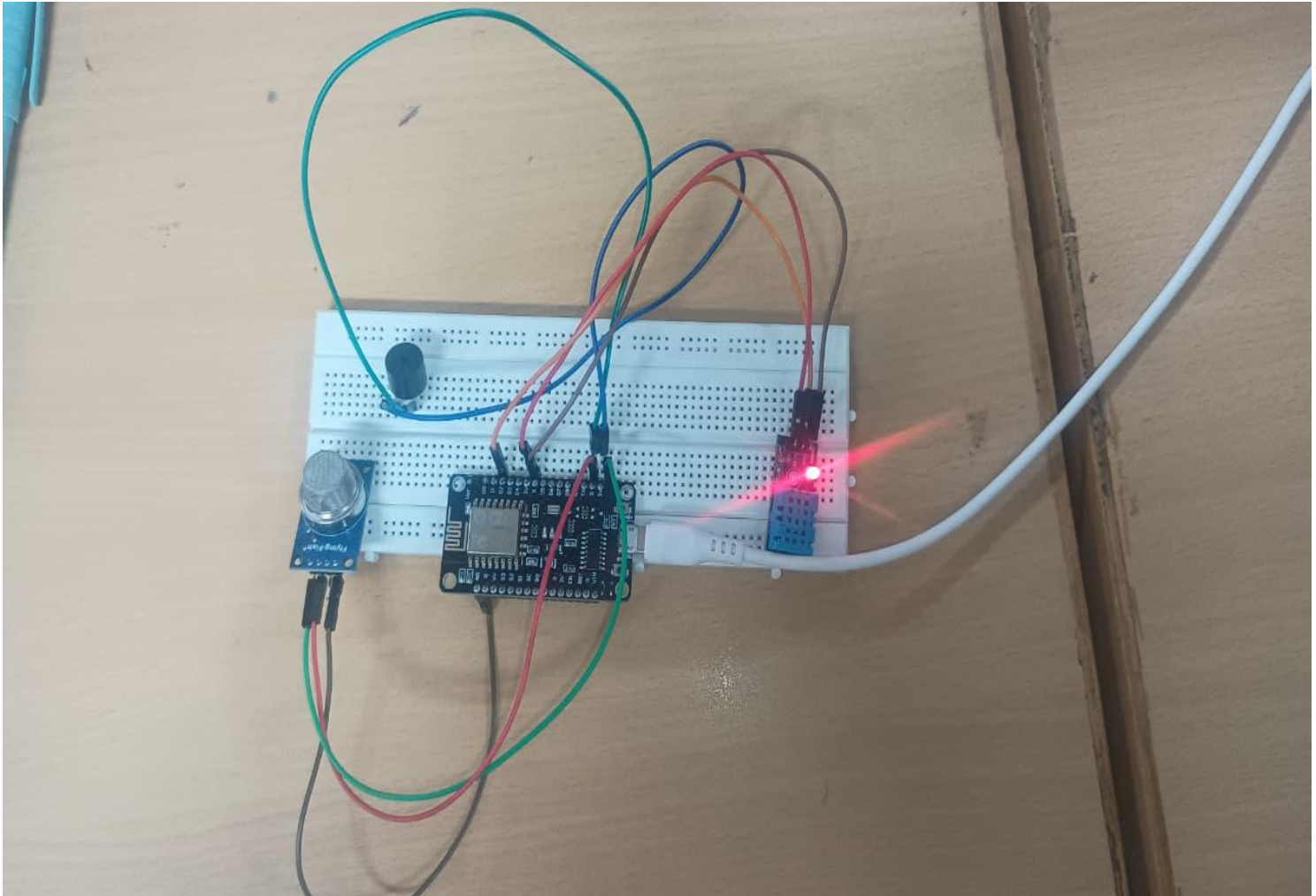
When the system is switched ON, the ESP32 initializes all connected sensors and establishes a Wi-Fi connection. The DHT11/DHT22 continuously measures temperature and humidity, while the MQ-135 provides an analog indication of changes in air quality.

The ESP32 processes these readings and checks them against selected threshold values. If the readings remain within the acceptable prototype range, the system indicates a **safe condition**. If an abnormal condition is detected, the buzzer and LED are activated.

## 10. Bill of Materials

| S.No | Component                  | Quantity | Approx. Cost |
|------|----------------------------|----------|--------------|
| 1    | ESP32 Development Board    | 1        | ₹350–500     |
| 2    | DHT11 Sensor               | 1        | ₹50–100      |
| 3    | MQ-135 Sensor              | 1        | ₹100–150     |
| 4    | Buzzer                     | 1        | ₹10–20       |
| 5    | LED                        | 1–2      | ₹5–10        |
| 6    | Resistors                  | Few      | ₹10          |
| 7    | Breadboard                 | 1        | ₹50–100      |
| 8    | Jumper Wires               | 1 set    | ₹50–100      |
| 9    | USB Cable                  | 1        | ₹50–100      |
| 10   | 5V Power Supply/Power Bank | 1        | ₹100–300     |

## 11. Prototype Implementation



The prototype can be constructed on a breadboard. The ESP32 is placed at the center and connected to the DHT11/DHT22 and MQ-135 sensors.

The sensors are positioned so that they can sense the surrounding industrial environment. The buzzer and LED are connected as warning indicators. The ESP32 is powered using a USB cable or suitable power source.

After programming the ESP32 using Arduino IDE, the system is tested by changing the surrounding temperature, humidity, and air-quality conditions. Sensor values are observed through the Serial Monitor and, after IoT integration, through the online dashboard.

## 12. Result and Performance Analysis

### Performance

- ✓ **Temperature monitoring:** Continuous.
- ✓ **Humidity monitoring:** Continuous.
- ✓ **Air-quality monitoring:** Continuous sensor-based indication.
- ✓ **Wireless monitoring:** Wi-Fi based.
- ✓ **Alert response:** Immediate after the programmed threshold is crossed.
- ✓ **Remote monitoring:** Possible through an IoT dashboard.
- ✓ **System operation:** Suitable for a small-scale prototype.

### Advantages

1. **Real-Time Monitoring** – Continuously monitors temperature, humidity, and air quality.
2. **Wireless Communication** – Sensor data can be transmitted through Wi-Fi without long communication cables.
3. **Remote Monitoring** – Industrial environmental conditions can be viewed remotely through an IoT dashboard.
4. **Early Warning** – Buzzer and LED provide alerts when parameters cross the set limits.
5. **Low Cost** – Uses affordable components such as ESP32, DHT11, and MQ-135.
6. **Easy to Install** – The prototype is simple to assemble and can be installed at different locations.
7. **Reduces Manual Inspection** – Continuous monitoring reduces the need for frequent manual checking.

### Limitations

1. **Wi-Fi Dependency** – Remote monitoring requires a stable Wi-Fi connection.
2. **Sensor Accuracy** – Low-cost sensors may not provide the accuracy of industrial-grade sensors.
3. **MQ-135 Calibration** – The air-quality sensor requires proper calibration for reliable measurements.
4. **Limited Sensor Range** – Sensors can monitor only the surrounding area where they are installed.

